



STORED PROCEDURES

en PostgreSQL

Workflows transaccionales en sistemas de producción

Base de Datos Avanzadas

UP Chiapas · Mtro. Ramsés Camas



github.com/RamsesCamas/BDA_Slides_And_Code

Evaluación del Curso

50%

Defensa oral individual

Demostrar competencia en vivo: escribir y explicar código SQL/PL sin ayuda

25%

Proyecto final con defensa

Proyecto integrador con preguntas individuales a cada miembro del equipo

15%

Quizzes en clase (pizarrón/oral)

Preguntas al azar al inicio de clase — sin dispositivos, sin aviso previo

10%

Labs entregados

Ejercicios prácticos de laboratorio entregados a tiempo



Mínimo 60 en defensa oral o se reprueba automáticamente, sin importar los demás componentes.

Agenda de Hoy



¿Qué es un Stored Procedure? Procedure vs Function

15 min



Sintaxis, parámetros y control de transacciones

15 min



¿Cuándo SÍ y cuándo NO usar Procedures?

10 min



4 ejemplos en producción real (10 → 1K usuarios)

35 min



Manejo de errores, buenas prácticas y anti-patrones

10 min



Ejercicio en clase con Docker

15 min

Escala de ejemplos:

10 users

40 users

100 users

1K users



Prerequisito: PL/pgSQL + Triggers de las clases anteriores. Docker Compose del repo levantado.

¿Qué es un Stored Procedure?

Un bloque de código SQL/PL almacenado en la BD que se invoca con CALL.



Analogía: Es como una receta de cocina guardada — la llamas por nombre y ejecuta todos los pasos en orden.



Se ejecuta con `CALL nombre_procedure(args)`



Puede hacer COMMIT y ROLLBACK (a diferencia de funciones)



Ideal para workflows multi-paso: insertar, validar, actualizar



NO retorna valor directamente — usa parámetros OUT o modifica tablas



Disponible desde PostgreSQL 11 (2018)



Antes de PG 11, todo se hacía con FUNCTION. Ahora los procedures existen para workflows transaccionales.

Procedure vs Function

Se parecen, pero tienen diferencias fundamentales:

Característica	FUNCTION	PROCEDURE
Se invoca con	SELECT fn()	CALL sp()
Retorna valor	SÍ (RETURNS tipo)	NO (usa OUT params)
COMMIT / ROLLBACK	✗ Prohibido	✓ Permitido
Usable en trigger	✓ Sí	✗ NO
Usable en SELECT	✓ SELECT fn()	✗ No se puede
Usable en CHECK / DEFAULT	✓ Sí	✗ NO
Caso de uso típico	Cálculos, transformar datos	Workflows multi-paso



Regla simple: ¿Necesitas COMMIT/ROLLBACK dentro del bloque? → PROCEDURE. ¿Necesitas retornar un valor o usarlo en SELECT? → FUNCTION.

Sintaxis Completa

CREATE PROCEDURE:

```
CREATE OR REPLACE PROCEDURE nombre(  
    IN p_param1 INT,  
    OUT p_resultado INT,  
    INOUT p_contador INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    -- Tu lógica aquí  
    COMMIT;  
    -- Permitido!  
END; $$;
```

Invocar con CALL:

```
-- Llamada simple  
CALL nombre(42, NULL, 0);  
  
-- Con variables (en PL/pgSQL)  
DECLARE  
    v_out INT; v_io INT := 5;  
BEGIN  
  
    CALL nombre(42, v_out, v_io);  
    RAISE NOTICE 'Resultado: %', v_out;  
END;
```

Tipos de parámetros:

Modo	Dirección	Uso
IN	Entrada →	Valor de solo lectura (default)
OUT	← Salida	Procedure escribe, caller lee
INOUT	↔ Ambos	Caller envía, procedure modifica



A diferencia de FUNCTION: no hay RETURNS. Los resultados salen por parámetros OUT o modificando tablas.

Control de Transacciones

LA razón principal para usar procedures en vez de functions:

PROCEDURE

```
CREATE PROCEDURE sp_ejemplo()  
LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO log VALUES('paso 1');  
  
    COMMIT; --  Guardado  
    INSERT INTO log VALUES('paso 2');  
  
    ROLLBACK; --  Deshecho  
END; $$;
```

FUNCTION

```
CREATE FUNCTION fn_ejemplo()  
RETURNS VOID  
LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO log VALUES('paso 1');  
  
    COMMIT; --  ERROR!  
    -- 'cannot commit while a  
    -- subtransaction is active'  
END; $$;
```



COMMIT dentro del procedure guarda los cambios HASTA ese punto.



ROLLBACK deshace solo lo que NO se ha commiteado aún.



Cada COMMIT inicia una nueva transacción implícita.



No funciona si CALL viene dentro de una transacción explícita (BEGIN...END).

¿Cuándo SÍ Usar Procedures?

Si la respuesta es SÍ, un procedure es probablemente la solución correcta:



Workflows multi-paso

¿Necesitas insertar en 3 tablas, validar y actualizar, todo como una operación?



Control transaccional fino

¿Quieres commitear pasos intermedios sin perder TODO si falla uno?



Lógica reutilizable del lado de

BD

¿Múltiples apps (web, mobile, script) deben ejecutar el mismo proceso?



Procesamiento batch/bulk

¿Necesitas procesar miles de registros con COMMIT cada N registros?



Encapsular reglas de negocio

críticas

¿La BD es la fuente de verdad para operaciones como cobros o inventario?



Pregunta clave: ¿este proceso NECESITA control transaccional intermedio? Si sí → PROCEDURE. Si no → FUNCTION.

¿Cuándo NO Usar Procedures?

Un procedure mal usado agrega complejidad sin beneficio:



Cálculos que retornan un valor

Si necesitas `SELECT calcular_total(id)`, eso es una `FUNCTION`.



Lógica usable en triggers

Los triggers solo aceptan `FUNCTIONS`. Un procedure no sirve aquí.



Operaciones simples de 1 paso

Un `INSERT` con validación simple no justifica la complejidad de un procedure.



Queries de solo lectura

Para reportes o consultas, usa `FUNCTION` con `RETURNS TABLE`.



Cuando la app ya maneja la transacción

Si tu framework (Django, Rails) controla `BEGIN/COMMIT`, no dupliques.



Si en duda entre `FUNCTION` y `PROCEDURE`: empieza con `FUNCTION`. Solo migra a `PROCEDURE` si necesitas `COMMIT/ROLLBACK`.

4 Sistemas en Producción Real

Procedures como orquestadores de workflows transaccionales:

10 users

Blog personal con CMS

Publicación masiva con COMMIT parcial

COMMIT parcial por iteración

40 users

Clínica dental

Registro de paciente multi-tabla

Multi-INSERT + EXCEPTION

100 users

E-commerce PyME

Procesamiento completo de pedido

FOR UPDATE + JSONB params

1,000 users

SaaS B2B

Migración de plan con prorrateo

Cross-table + cálculo
prorrateo



Patrón común: el procedure ORQUESTA la operación, los triggers REACCIONAN a los cambios que genera.

Blog — Publicación Masiva

10 users

Publicar todos los borradores de un autor con COMMIT individual por artículo.

Problema: El admin quiere publicar 20 borradores. Si uno falla, NO debe cancelar los otros 19.

```
CREATE OR REPLACE PROCEDURE sp_publicar_borradores(IN p_autor VARCHAR, OUT p_publicados INT)
LANGUAGE plpgsql AS $$
DECLARE
    rec RECORD;
BEGIN
    p_publicados := 0;
    FOR rec IN SELECT id, titulo FROM articulos
        WHERE autor = p_autor AND publicado = false
    LOOP
        BEGIN
            UPDATE articulos SET publicado = true,
                published_at = now() WHERE id = rec.id;
            p_publicados := p_publicados + 1;
            COMMIT; -- Guarda ESTE artículo
        EXCEPTION WHEN OTHERS THEN
            ROLLBACK; -- Solo pierde ESTE
            RAISE NOTICE 'Error en: %', rec.titulo;
        END;
    END LOOP;
END; $$;

CALL sp_publicar_borradores('ramses', NULL);
```



Imposible con FUNCTION: COMMIT dentro de un loop. Cada iteración es una transacción independiente.

Clínica — Registro de Paciente

40 users

Crear paciente + expediente + primera cita en una sola operación atómica.

Problema: Si se crea el paciente pero falla la cita, queda un registro huérfano. Debe ser todo o nada.

```
CREATE OR REPLACE PROCEDURE sp_registrar_paciente(  
  IN p_nombre VARCHAR, IN p_curp VARCHAR, IN p_tel VARCHAR,  
  OUT p_paciente_id INT  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
  -- 1) Crear paciente  
  INSERT INTO pacientes(nombre, curp, telefono)  
  VALUES(p_nombre, p_curp, p_tel) RETURNING id INTO p_paciente_id;  
  
  -- 2) Crear expediente médico vacío  
  INSERT INTO expedientes(paciente_id, fecha_apertura)  
  VALUES(p_paciente_id, now());  
  
  -- 3) Agendar primera cita  
  INSERT INTO citas(paciente_id, tipo, fecha)  
  VALUES(p_paciente_id, 'PRIMERA_VEZ', now() + interval '7 days');  
  
  COMMIT;  
EXCEPTION WHEN OTHERS THEN  
  ROLLBACK; RAISE;  
END; $$;
```



RETURNING id INTO captura el ID generado. Si la cita falla, ROLLBACK deshace paciente y expediente también.

E-commerce — Procesar Pedido

100 users

Crear pedido, validar stock, descontar inventario y calcular total.

Problema: El pedido tiene múltiples items (JSON). Cada item debe validarse y descontarse atómicamente.

```
CREATE OR REPLACE PROCEDURE sp_procesar_pedido(
  IN p_cliente INT, IN p_items JSONB, OUT p_pedido_id INT, OUT p_total NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE item RECORD; v_stock INT; v_precio NUMERIC;
BEGIN
  INSERT INTO pedidos(cliente_id, estado) VALUES(p_cliente, 'PROCESANDO')
  RETURNING id INTO p_pedido_id;
  p_total := 0;
  FOR item IN SELECT * FROM jsonb_to_recordset(p_items) AS x(producto_id INT, cantidad INT)
  LOOP
    SELECT stock, precio INTO v_stock, v_precio
    FROM productos WHERE id = item.producto_id FOR UPDATE;
    IF v_stock < item.cantidad THEN
      RAISE EXCEPTION 'Stock insuficiente producto %', item.producto_id;
    END IF;
    INSERT INTO pedido_items(pedido_id, producto_id, cantidad, precio_unitario)
    VALUES(p_pedido_id, item.producto_id, item.cantidad, v_precio);
    UPDATE productos SET stock = stock - item.cantidad WHERE id = item.producto_id;
    p_total := p_total + (v_precio * item.cantidad);
  END LOOP;
  UPDATE pedidos SET total = p_total, estado = 'CONFIRMADO' WHERE id = p_pedido_id;
  COMMIT;
EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END; $$;
```



jsonb_to_recordset() convierte JSON a filas iterables. FOR UPDATE bloquea stock durante el proceso.

SaaS B2B — Migrar Plan

1,000 users

Upgrade/downgrade con prorrateo y registro de cambio.

Problema: Al cambiar de plan se debe calcular crédito restante, aplicar nuevo precio y registrar auditoría.

```
CREATE OR REPLACE PROCEDURE sp_migrar_plan(
  IN p_user_id INT, IN p_nuevo_plan_id INT,
  OUT p_credito NUMERIC, OUT p_prox_cobro DATE
)
LANGUAGE plpgsql AS $$
DECLARE v_sub RECORD; v_dias_rest INT; vPrecio_viejo NUMERIC; vPrecio_nuevo NUMERIC;
BEGIN
  -- 1) Obtener suscripción actual
  SELECT s.*, p.precio INTO v_sub, vPrecio_viejo FROM suscripciones s
  JOIN planes p ON s.plan_id = p.id
  WHERE s.usuario_id = p_user_id AND s.activa FOR UPDATE;
  -- 2) Calcular crédito prorrateado
  v_dias_rest := v_sub.fecha_fin - CURRENT_DATE;
  p_credito := ROUND((vPrecio_viejo / 30.0) * v_dias_rest, 2);
  SELECT precio INTO vPrecio_nuevo FROM planes WHERE id = p_nuevo_plan_id;
  -- 3) Actualizar suscripción + registrar cambio
  UPDATE suscripciones SET plan_id = p_nuevo_plan_id,
    fecha_inicio = CURRENT_DATE, fecha_fin = CURRENT_DATE + 30 WHERE id = v_sub.id;
  INSERT INTO cambios_plan(user_id, plan_anterior, plan_nuevo, credito, fecha)
  VALUES(p_user_id, v_sub.plan_id, p_nuevo_plan_id, p_credito, now());
  p_prox_cobro := CURRENT_DATE + 30;
  COMMIT;
EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END; $$;
```



FOR UPDATE en la suscripción evita que dos requests cambien el plan simultáneamente.

Patrones por Escala: Resumen

Procedures como orquestadores — la complejidad crece con la escala:

Escala	Patrón	COMMIT	Tablas	Clave técnica
10 users	Batch con loop	Por iteración	1	COMMIT parcial en loop
40 users	Multi-tabla atómico	Al final	3	RETURNING INTO + EXCEPTION
100 users	JSONB + FOR UPDATE	Al final	3	jsonb_to_recordset + lock
1K users	Prorratio + audit	Al final	3	Cálculo intermedio + cross-join



Patrón emergente: A mayor escala, el procedure hace más validaciones previas (SELECT ... FOR UPDATE), más registros de auditoría, y la lógica de negocio dentro del procedure se vuelve más compleja. El EXCEPTION block se vuelve crítico.

Procedure + Trigger = El procedure ORQUESTA (CALL sp_procesar_pedido). Los triggers REACCIONAN (trg_descontar_stock, trg_audit). Cada uno hace lo suyo.

Manejo de Errores

EXCEPTION es tu red de seguridad dentro de procedures:

```
CREATE OR REPLACE PROCEDURE sp_con_erroses()
LANGUAGE plpgsql AS $$
BEGIN

-- Operaciones normales...

INSERT INTO ordenes VALUES(...);
COMMIT;

EXCEPTION
  WHEN unique_violation THEN
    ROLLBACK;
    RAISE NOTICE 'Registro duplicado';
  WHEN foreign_key_violation THEN
    ROLLBACK;
    RAISE NOTICE 'Referencia inválida';
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE NOTICE 'Error: % - %', SQLSTATE, SQLERRM;
END; $$;
```

Errores comunes:

Nombre	Código
unique_violation	23505
foreign_key_violation	23503
not_null_violation	23502
check_violation	23514
raise_exception	P0001
OTHERS	(todos)

Variables de diagnóstico:

Variable	Contenido
SQLSTATE	Código de error (23505)
SQLERRM	Mensaje legible



WHEN OTHERS es el catch-all. SIEMPRE incluye ROLLBACK + log del error, o los datos quedan inconsistentes.

Buenas Prácticas

1

Prefijo sp_ consistente

sp_registrar_paciente, sp_procesar_pedido. Distingue de funciones (fn_).

2

Siempre EXCEPTION block

Sin EXCEPTION, un error deja la transacción en estado indeterminado.

3

Documenta parámetros OUT

COMMENT ON PROCEDURE sp_x IS 'OUT: p_id = ID generado, p_total = ...'.

4

Valida ANTES de modificar

Primero SELECT + validaciones. Luego INSERT/UPDATE. Reduce rollbacks.

5

Un procedure = un workflow

Si hace dos cosas no relacionadas, divide en dos procedures.

6

COMMIT explícito al final

Aunque PG hace auto-commit, ser explícito documenta la intención.

Anti-Patrones: Lo Que NO Hacer



Procedure de 500+ líneas

Si es tan largo, divide en sub-procedures o funciones auxiliares.



COMMIT después de cada INSERT

Pierdes atomicidad. Si el paso 3 falla, pasos 1 y 2 ya se commitearon.



Ignorar EXCEPTION

Sin manejo de errores, un fallo deja datos a medias sin que nadie lo sepa.



Procedure para SELECT de solo lectura

Eso es un FUNCTION con RETURNS TABLE. Procedure sin DML no tiene sentido.



CALL dentro de transacción explícita

BEGIN; CALL sp_x(); COMMIT; — el COMMIT dentro del SP falla. Elige uno.



CALL sp_x() desde psql/app funciona. CALL sp_x() dentro de BEGIN...END explícito → error si el SP hace COMMIT.

Administrar Procedures

Ver procedures del schema:

```
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'public'
  AND routine_type = 'PROCEDURE';
```

Ver código fuente:

```
SELECT prosrc FROM pg_proc
WHERE proname = 'sp_registrar_paciente';
```

Modificar (CREATE OR REPLACE):

```
CREATE OR REPLACE PROCEDURE sp_nombre(...)
LANGUAGE plpgsql AS $$ ... $$;
```

Eliminar:

```
DROP PROCEDURE sp_registrar_paciente(VARCHAR, VARCHAR, VARCHAR);
```



DROP PROCEDURE requiere los tipos de parámetros para identificar cuál borrar (overloading posible).



Ejercicio en Clase

1. Crear tablas: clientes, productos, pedidos, pedido_items
2. Procedure `sp_crear_cliente(IN nombre, IN email, OUT id)` — insertar cliente
3. Procedure `sp_procesar_pedido(IN cliente_id, IN items JSONB, OUT pedido_id)`
4. Dentro: validar stock, crear pedido, descontar inventario, calcular total
5. Manejo de errores: si stock insuficiente, ROLLBACK y mensaje claro
6. Probar: CALL con JSON de 3 productos, verificar pedido y stock



github.com/RamsesCamas/BDA_Slides_And_Code

Base de Datos Avanzadas · UP Chiapas · Mtro. Ramsés Camas

Resumen: Lo Que Aprendimos Hoy

`CREATE PROCEDURE ... LANGUAGE plpgsql`

Define un workflow almacenado en la BD

`CALL sp_nombre(args)`

Invoca el procedure (no SELECT)

`IN / OUT / INOUT`

Parámetros de entrada, salida o ambos

`COMMIT / ROLLBACK`

Control transaccional dentro del procedure

`EXCEPTION WHEN ... THEN`

Manejo de errores con rollback selectivo

`RETURNING id INTO variable`

Capturar IDs generados por INSERT

`jsonb_to_recordset()`

Iterar sobre parámetros JSON estructurados

`SQLSTATE / SQLERRM`

Diagnóstico de errores dentro de EXCEPTION

Siguiente clase: Cursores + Optimización de Querías (necesitas procedures como base).